

# AI2 Assignment 1 — Report

## D4-V2: Search and Rescue — Unknown Victim Location

PDDL · PDDL+ · Search and Rescue · Unknown Victim Location · Health Degradation · ENHSP

---

### Abstract

---

This project models a search-and-rescue scenario in which a single mobile rescue robot operates inside a damaged building. The building topology is known, but the victim location is not directly available to the robot. The robot must explore the environment, inspect rooms, detect the victim, and perform rescue operations.

The assignment is implemented in two required parts. Q1 uses Basic PDDL and approximates the unknown victim location problem through explicit symbolic inspection predicates. Q2 extends the model with PDDL+ constructs, introducing time-dependent victim health degradation through a numeric fluent, a continuous process, and failure events.

The final repository also contains two additional versions. These are not duplicates of the same model: they document a modelling ambiguity in the word `rescue`. In the core assignment, `rescue` is interpreted as an on-site rescue action. In the transport variant, `rescue` means evacuating the patient to the base. In the definitive model, the robot assesses the patient and chooses the appropriate rescue branch depending on the current health value.

---

## 1. Assignment Scope

---

The selected theme is:

D4-V2: Search and Rescue — Unknown Victim Location

The scenario requires a rescue robot to operate in a damaged building where the topology is known but the victim location is initially unknown. The robot must explore rooms, inspect them, detect the victim, and then perform rescue operations.

The official assignment requirements are summarized below.

| Part          | Required modelling element                                 | Implemented in                                                              |
|---------------|------------------------------------------------------------|-----------------------------------------------------------------------------|
| Q1 Basic PDDL | Approximate exploration using explicit predicates          | <code>codes/D4-V2_search_and_rescue/Q1_basic_pddl/</code>                   |
| Q1 Basic PDDL | Provide one known-location instance                        | <code>problem_known_location.pddl</code>                                    |
| Q1 Basic PDDL | Provide one exploration instance                           | <code>problem_exploration.pddl</code>                                       |
| Q1 Basic PDDL | Provide valid plans                                        | <code>plan_known_location.txt</code> ,<br><code>plan_exploration.txt</code> |
| Q2 PDDL+      | Model victim health degradation                            | <code>domain_plus.pddl</code>                                               |
| Q2 PDDL+      | Introduce failure/discovery event logic                    | <code>victim-dies</code> event and detection logic                          |
| Q2 PDDL+      | Show that exploration delay affects rescue success         | fast and delayed rescue instances                                           |
| Discussion    | Explain limits of classical PDDL for partial observability | <code>notes/discussion_partial_observability.md</code>                      |
| Discussion    | Explain interaction between sensing and time               | <code>notes/discussion_sensing_time.md</code>                               |

## 2. Planning Background Used in the Project

A planning problem is described by an initial state, a set of actions that may change the state, and a goal condition. In PDDL, this separation is encoded through two files:

| File         | Role in the project                                                                                   |
|--------------|-------------------------------------------------------------------------------------------------------|
| Domain file  | Defines the general logic of the world: types, predicates, functions, actions, processes, and events. |
| Problem file | Defines the concrete instance: objects, initial state, numeric values, and goal condition.            |
| Plan file    | Stores the action sequence returned by the planner, or documents the expected failure case.           |

This project uses this structure consistently. The domain describes what a rescue robot can do in general, while each problem file changes the initial conditions, such as whether the victim location is already known, whether exploration is required, or whether the mission starts too late.

Classical PDDL is used for Q1 because the first part of the assignment asks for a Basic PDDL model. PDDL+ is used for Q2 because victim health is not only changed by discrete actions: it decreases over time while the mission is active.

### 3. Why the Repository Contains Three Versions

The repository intentionally contains three related versions of the same search-and-rescue scenario.

The reason is that the assignment uses the word `rescue`, but this word is not completely unambiguous. During modelling, three reasonable interpretations emerged:

1. `rescue` = reach the victim and perform an on-site rescue action
2. `rescue` = evacuate the patient to a safe base
3. `rescue` = assess the patient and choose direct transport or stabilization

Instead of hiding this ambiguity, the repository keeps all three versions and documents how each interpretation changes the planning model.

| Version           | Folder                                                 | Interpretation of <code>rescue</code>                                                                          | Role                                                           |
|-------------------|--------------------------------------------------------|----------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------|
| Core assignment   | <code>codes/D4-V2_search_and_rescue/</code>            | Rescue is an on-site action after detection.                                                                   | Required Q1/Q2 deliverable.                                    |
| Transport variant | <code>codes/D4-V2_search_and_rescue_variant/</code>    | Rescue means loading the patient, transporting them to base, and unloading them there.                         | Extra model exploring a stronger interpretation.               |
| Definitive model  | <code>codes/D4-V2_search_and_rescue_definitive/</code> | Rescue depends on patient health: direct transport if stable, stabilization first if low, failure if too late. | Final extended model and recommended version to inspect first. |

This is the main design rationale behind the final structure. The three versions are not redundant: they represent three different levels of modelling depth.

### 4. Repository Structure

```

AI2_Assignment1/
├── codes/
│   ├── D4-V2_search_and_rescue/
│   │   ├── Q1_basic_pddl/
│   │   ├── Q2_pddl_plus/
│   │   ├── notes/
│   │   └── README.md
│   ├── D4-V2_search_and_rescue_variant/
│   │   ├── notes/
│   │   └── README.md
│   └── D4-V2_search_and_rescue_definitive/
│       ├── notes/
│       └── README.md
└── Report/

```

```
├─ slide/
└─ README.md
```

The core assignment folder contains the official Q1 and Q2 material. The variant and definitive folders are additional modelling extensions.

## 5. Core Assignment Model

### 5.1 Scenario Representation

The core model represents the damaged building as a graph of rooms. Movement is possible only when two rooms are connected. The robot has a current location, rooms may be inspected, and the victim may be detected only after inspection.

| Concept              | PDDL representation          | Reason                                                   |
|----------------------|------------------------------|----------------------------------------------------------|
| Robot position       | <code>robot-at</code>        | Tracks where the rescue robot currently is.              |
| Topology             | <code>connected</code>       | Encodes the known building map.                          |
| Inspection           | <code>inspected</code>       | Makes exploration explicit.                              |
| Real victim location | <code>victim-at</code>       | Encodes the true model state required by classical PDDL. |
| Detected information | <code>victim-detected</code> | Represents what the robot has discovered.                |
| Rescue status        | <code>rescued</code>         | Goal predicate for successful rescue.                    |

The important modelling decision is the separation between the real state and the robot's acquired information.

```
;; Real world fact encoded in the problem file.
;; Classical PDDL requires the full initial state to be explicitly represented.
(victim-at infirmary)

;; Information fact acquired by the robot after inspecting the correct room.
;; The rescue action depends on this predicate, not directly on victim-at.
(victim-detected infirmary)
```

This does not create true partial observability, but it prevents the robot from acting as if it already knew the victim location.

### 5.2 Q1 — Basic PDDL

Q1 approximates exploration using classical PDDL predicates and actions.

| File                        | Purpose                                                                                   |
|-----------------------------|-------------------------------------------------------------------------------------------|
| domain.pddl                 | Defines movement, inspection, detection, and rescue actions.                              |
| problem_known_location.pddl | A simpler instance where the victim location is known or directly reachable in the model. |
| problem_exploration.pddl    | An instance where the robot must inspect rooms before rescue.                             |
| plan_known_location.txt     | Saved plan for the known-location case.                                                   |
| plan_exploration.txt        | Saved plan for the exploration case.                                                      |

The Q1 causal chain is:

move → inspect → detect → rescue

The planner is not given a procedural script. Instead, valid action order emerges from preconditions. For example, rescue can only happen after detection, and detection can only happen after the robot reaches and inspects the correct room.

| Action              | Main role                               | Modelling reason                   |
|---------------------|-----------------------------------------|------------------------------------|
| move                | Move between connected rooms.           | Encodes topological navigation.    |
| inspect-empty-room  | Inspect a room without the victim.      | Allows exploration of wrong rooms. |
| inspect-victim-room | Inspect the room containing the victim. | Produces the detection predicate.  |
| rescue              | Complete the on-site rescue.            | Requires prior victim detection.   |

### 5.3 Why Q1 Is Only an Approximation

The victim location is described as unknown in the natural-language scenario. However, classical PDDL does not directly model unknown hidden state. A classical PDDL problem needs a complete initial state, so the real victim location must still appear in the problem file.

The model handles this by separating:

| Layer           | Predicate            | Meaning                                              |
|-----------------|----------------------|------------------------------------------------------|
| World reality   | victim-at ?loc       | The victim is actually located at ?loc.              |
| Robot knowledge | victim-detected ?loc | The robot has discovered that the victim is at ?loc. |

The rescue action depends on `victim-detected`, not on `victim-at`. Therefore, the planner cannot rescue the victim before the inspection sequence has generated the symbolic detection fact.

This is a controlled abstraction. It is useful for Q1 because the assignment explicitly allows exploration to be approximated with explicit predicates, but it is not equivalent to a POMDP, a belief-state planner, or a realistic perception pipeline.

## 6. Q2 — PDDL+ Model

Q2 extends the core search-and-rescue problem with victim health degradation.

The key difference is that the world changes even while the robot is moving or waiting. In Q1, time is not represented. In Q2, delays matter because the victim health decreases continuously.

### 6.1 Numeric Fluent

The model introduces the numeric fluent:

```
;; Numeric fluent used to represent the patient's current health level.
;; Higher values mean more remaining time before mission failure.
(victim-health)
```

This changes the problem from simple reachability to feasibility under time pressure. The question is no longer only whether the robot can eventually reach the victim, but whether it can do so before health reaches zero.

### 6.2 Health Degradation Process

Victim health is modelled as a PDDL+ process.

```
(:process health-degradation
  :parameters ()
  :precondition (and
    ;; The process is active only during the mission.
    (mission-active)
    ;; Health decreases only while the victim is alive.
    (victim-alive)
    ;; The process stops once health has reached zero.
    (> (victim-health) 0)
  )
  :effect (and
    ;; #t represents elapsed time, so health decreases continuously over time.
    (decrease (victim-health) (* #t 1))
  )
)
```

The term `#t` is the essential PDDL+ element here. It means that the amount of health lost depends on elapsed time, not on the number of discrete actions alone.

## 6.3 Failure Event

Failure is represented as an automatic event.

```
(:event victim-dies
  :parameters ()
  :precondition (and
    ;; The event can fire only during an active mission.
    (mission-active)
    ;; The event is relevant only if the victim has not already died.
    (victim-alive)
    ;; The threshold condition triggers failure.
    (<= (victim-health) 0)
  )
  :effect (and
    ;; The world records the failed state.
    (victim-dead)
    ;; The victim is no longer alive.
    (not (victim-alive))
    ;; The mission is stopped because rescue is no longer possible.
    (not (mission-active))
  )
)
```

The planner does not choose this event. The event is triggered by the world when the threshold condition becomes true.

## 6.4 Fast Rescue vs Delayed Rescue

Q2 contains two main cases.

| Case           | Initial idea                                                 | Expected result                                       |
|----------------|--------------------------------------------------------------|-------------------------------------------------------|
| Fast rescue    | The robot moves, inspects, detects, and rescues quickly.     | Rescue succeeds before health reaches zero.           |
| Delayed rescue | The problem forces an explicit waiting period before rescue. | Rescue should fail because health reaches zero first. |

In the fast rescue problem, the intended sequence is:

```
start-move entrance corridor
start-move corridor lab
start-move lab infirmary
start-inspect-victim-room infirmary
start-rescue infirmary
```

The total time is lower than the initial health threshold, so the victim remains alive.

In the delayed rescue problem, the model forces a wait before rescue. The intended timing analysis is:

| Step | Duration |
|------|----------|
| wait | 5        |

| Step                     | Duration  |
|--------------------------|-----------|
| move entrance → corridor | 2         |
| move corridor → lab      | 2         |
| move lab → infirmary     | 2         |
| inspect victim room      | 1         |
| rescue                   | 1         |
| <b>Total</b>             | <b>13</b> |

With initial health equal to 10 and degradation rate equal to 1 per time unit, rescue should not be possible after 13 time units. The delayed case is therefore interpreted as a model-level failure caused by time pressure.

## 7. Transport-to-Base Variant

The first optional extension addresses the ambiguity of `rescue`.

In the core model, rescue is completed on site. This is acceptable for the assignment, but in many rescue scenarios the natural interpretation is stronger: the patient must be evacuated to a safe base.

The variant therefore changes the goal condition.

| Core model                                 | Transport variant                                               |
|--------------------------------------------|-----------------------------------------------------------------|
| Rescue is completed at the victim room.    | Rescue requires the patient to be transported to base.          |
| Goal is essentially <code>rescued</code> . | Goal requires both rescue status and patient location at base.  |
| No patient-carrying navigation is needed.  | The robot must load, move with the patient, and unload at base. |

The variant adds transport-specific actions:

| Action                                    | Purpose                                   |
|-------------------------------------------|-------------------------------------------|
| <code>start-load-patient</code>           | Load the detected patient onto the robot. |
| <code>start-move-with-patient</code>      | Move while carrying the patient.          |
| <code>start-unload-patient-at-base</code> | Complete evacuation at the safe base.     |

This version is not a replacement for the required Q1/Q2 deliverable. It is an additional model that explores a richer interpretation of the same assignment wording.



## 8. Definitive Model

The definitive model is the most complete version of the project.

It combines:

| Feature               | Purpose                                         |
|-----------------------|-------------------------------------------------|
| Search and inspection | Preserve the unknown-location structure.        |
| Victim detection      | Ensure rescue depends on acquired information.  |
| Health degradation    | Model time pressure continuously.               |
| Patient assessment    | Let the robot classify the patient's condition. |
| Direct transport      | Used when health is high enough.                |
| Stabilization         | Used when health is low but still recoverable.  |
| Mission deadline      | Prevents unrealistic planner waiting.           |
| Too-late instance     | Demonstrates correct unsolvability.             |

The intended high-level pipeline is:

```
search → inspect → detect victim → assess patient → choose branch → transport to base
```

### 8.1 Health-Based Branch Selection

The branch is selected using the numeric value of `victim-health` at assessment time.

| Health situation                   | Planner branch           | Expected behaviour                                       |
|------------------------------------|--------------------------|----------------------------------------------------------|
| Health high enough                 | Direct transport         | The robot loads the patient and transports them to base. |
| Health low but recoverable         | Stabilize then transport | The robot stabilizes the patient before evacuation.      |
| Health too low or mission too late | No valid rescue plan     | The problem becomes unsolvable.                          |

This branch selection is not implemented as an external `if` statement. It is encoded declaratively through PDDL/PDDL+ preconditions and effects. The planner selects a valid action sequence because only some branches are applicable in the current state.

### 8.2 Why Stabilization Was Added

The stabilization branch is an extra modelling layer beyond the original assignment. It addresses a realistic rescue scenario: when patient health is too low, immediate transport may be unsafe or impossible, so the robot must stabilize the patient before moving.

In modelling terms, stabilization acts as a state transition that changes the available rescue strategy. It prevents the definitive model from being a simple transport variant and turns it into a health-aware planning problem.

### 8.3 Why the Mission Deadline Was Added

During testing, the planner could sometimes exploit waiting behaviour in undesirable ways. Since health decreases over time, arbitrary waiting could force the health value below a threshold and make the planner select stabilization even when direct transport would have been the natural immediate branch.

The definitive model therefore includes a mission clock and a mission deadline. This constrains the plan and prevents unrealistic waiting from becoming a modelling artefact.

### 8.4 Definitive Test Outcomes

| Problem file                                         | Expected outcome                            | Diagnostic check                                |
|------------------------------------------------------|---------------------------------------------|-------------------------------------------------|
| problem_definitive_direct_transpo<br>rt.pddl         | Solved with direct transport.               | start-stabilize-patient should not appear.      |
| problem_definitive_stabilize_then<br>_transport.pddl | Solved with stabilization before transport. | start-stabilize-patient should appear.          |
| problem_definitive_too_late.pddl                     | Unsolvable.                                 | Rescue is impossible before failure conditions. |

This final version is the recommended one to inspect first because it documents the richest interpretation of the project.

## 9. Planner and Validation Notes

The main planner used during the project was ENHSP, executed locally through Docker for the PDDL+ parts.

| Aspect                | Validation approach                                                           |
|-----------------------|-------------------------------------------------------------------------------|
| Q1 syntax             | Checked with PDDL tooling and planner execution.                              |
| Q1 plans              | Saved plans for known-location and exploration instances.                     |
| Q2 health degradation | Checked through fast and delayed rescue cases.                                |
| Transport variant     | Tested as an additional PDDL+ model.                                          |
| Definitive model      | Tested on direct transport, stabilize-then-transport, and too-late instances. |

A planner-output issue was observed in older delayed PDDL+ runs: the online ENHSP service returned an empty plan for a non-empty goal in one delayed instance. Since the

goal predicates were not true in the initial state, that empty plan was not semantically valid. The report and notes therefore interpret that case at the model level: the delayed rescue should fail because the victim dies before rescue can be completed.

This is an important distinction. The project does not treat every planner output as automatically meaningful. Planner output is checked against the model semantics.

## 10. Discussion: Partial Observability

The assignment scenario is naturally partially observable: the robot does not know where the victim is before exploration. A realistic robot would need to maintain a belief over possible victim locations and update this belief using sensor observations.

The Q1 model does not implement this. Classical PDDL assumes that the initial state is explicitly represented, so the true victim location must be encoded in the problem file. The project handles this limitation by separating world truth from robot knowledge.

| Real robotic concept    | Approximation used in Q1                                             |
|-------------------------|----------------------------------------------------------------------|
| Hidden victim location  | <code>victim-at ?loc</code> encoded in the full symbolic state.      |
| Sensor observation      | <code>inspect-empty-room</code> / <code>inspect-victim-room</code> . |
| Information acquisition | <code>victim-detected ?loc</code> .                                  |
| Belief update           | Not implemented; approximated by deterministic predicates.           |
| Uncertainty             | Not modelled probabilistically.                                      |

This is acceptable for the Basic PDDL part because the assignment asks for an approximation using explicit predicates. However, it remains a limitation of the model.

A full treatment of the scenario would require a belief-space model or a POMDP-like representation. That would allow the robot to reason over possible victim locations under uncertainty, but it is outside the required scope of this assignment.

## 11. Discussion: Sensing and Time

Q2 adds the most important interaction in the scenario: sensing consumes time, and time affects survival.

In Q1, inspection only changes symbolic knowledge. In Q2, inspection also delays rescue. Because victim health decreases continuously, every move, inspection, wait, or transport action has an indirect effect on mission feasibility.

| Activity     | Symbolic effect              | Time-related consequence                     |
|--------------|------------------------------|----------------------------------------------|
| Moving       | Changes robot location.      | Health continues to decrease.                |
| Inspecting   | Produces information.        | Health continues to decrease.                |
| Waiting      | Produces delay.              | Health decreases without progress.           |
| Transporting | Moves patient/base relation. | Health may still decrease unless stabilized. |
| Stabilizing  | Changes patient condition.   | Enables safer or feasible continuation.      |

The important modelling result is that exploration is no longer free. A longer exploration path may make rescue impossible even if a valid symbolic path exists.

This is the core reason for using PDDL+: the planner must reason about trajectories over time, not only about discrete states.

## 12. Extra Work Beyond the Official Assignment

The repository includes several elements that go beyond the minimum assignment requirements.

| Extra element           | Why it was added                                                                            |
|-------------------------|---------------------------------------------------------------------------------------------|
| Three-version structure | Documents the ambiguity of the word <code>rescue</code> .                                   |
| Transport-to-base model | Tests a stronger evacuation interpretation.                                                 |
| Definitive model        | Combines detection, health, transport, stabilization, and failure.                          |
| Patient assessment      | Makes the rescue strategy depend on the current numeric health value.                       |
| Stabilization branch    | Adds a medically motivated alternative to immediate transport.                              |
| Mission deadline        | Prevents artificial planner waiting from distorting branch selection.                       |
| Too-late problem        | Demonstrates unsolvability when rescue is no longer feasible.                               |
| Detailed notes          | Separates modelling choices, partial observability discussion, and sensing/time discussion. |

These additions are useful because they show the modelling process, not just the final files. The project does not only answer the assignment; it also records the design decisions and alternative interpretations considered during development.

## 13. Limitations

The final project is still an abstraction.

| Limitation                    | Explanation                                                               |
|-------------------------------|---------------------------------------------------------------------------|
| No real partial observability | Unknown location is approximated with symbolic predicates.                |
| No probabilistic sensing      | Inspection is deterministic.                                              |
| No belief state               | The robot does not maintain probabilities over possible victim locations. |
| No geometric navigation       | Rooms and connections are topological, not metric.                        |
| No motion planning            | Movement assumes traversability once <code>connected</code> is true.      |
| No execution uncertainty      | Actions are assumed to succeed if preconditions hold.                     |
| Simplified health model       | Health decreases linearly; real medical dynamics are more complex.        |
| Planner-dependent behaviour   | Some PDDL+ outputs require semantic interpretation and validation.        |

These limitations are not accidental. They are consequences of choosing a tractable PDDL/PDDL+ abstraction for an assignment-scale planning model.

## 14. Conclusion

The project implements the D4-V2 Search and Rescue assignment using a progressive modelling approach.

The core model satisfies the official requirements: it provides a Basic PDDL approximation of exploration under unknown victim location, and a PDDL+ extension where victim health decreases over time. The notes explain why classical PDDL cannot fully represent partial observability and why PDDL+ is needed to capture the interaction between sensing delay and rescue success.

The additional transport and definitive versions document the central modelling ambiguity around the word `rescue`. If rescue is interpreted minimally, the core model is sufficient. If rescue means evacuation, transport actions are required. If rescue depends on the patient's condition, the model must include assessment, stabilization, and health-based branch selection.

The definitive version is therefore the most complete model: it preserves the original exploration requirement, includes PDDL+ health degradation, transports the patient to

base, and chooses between direct transport and stabilization depending on the state reached during planning.

## Appendix A — Source Notes Used

| Note file                                                                                   | Main content used in this report                                                         |
|---------------------------------------------------------------------------------------------|------------------------------------------------------------------------------------------|
| <code>codes/D4-V2_search_and_rescue/notes/modelling_choices.md</code>                       | Scenario, core modelling decisions, Q1 approximation, Q2 extension.                      |
| <code>codes/D4-V2_search_and_rescue/notes/discussion_partial_observability.md</code>        | Separation between victim-at and victim-detected ; limits of classical PDDL.             |
| <code>codes/D4-V2_search_and_rescue/notes/discussion_sensing_time.md</code>                 | Health degradation, #t , failure event, fast vs delayed rescue timing.                   |
| <code>codes/D4-V2_search_and_rescue_variant/notes/variant_transport_to_base_notes.md</code> | Motivation for interpreting rescue as evacuation.                                        |
| <code>codes/D4-V2_search_and_rescue_definitive/notes/</code>                                | Health-based branch selection, stabilization, mission deadline, and too-late validation. |